SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY

COURSE CODE: CSA5108

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS: 5 **Total Marks: 100**

Annexure A

Code of Conduct:

*I **A.Sai Joshitha(192424220)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: A.Sai Joshitha

Annexure A
Debugging & Optimisation Task

QUESTIONS:

1. Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)A Python program implementing DES and 3DES encryption produces inconsistentciphertext for identical inputs across multiple runs.

A. Identify and fix logical and implementation errors in key scheduling and permutation steps.

B. Optimize the code to reduce execution time for large input datasets (≥10 MB).

C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

4. Optimization and Security Analysis of Diffie-Hellman Key Exchange

A Python implementation of the Diffie-Hellman key exchange is vulnerable to performance bottlenecks and potential security flaws.

A. Debug incorrect shared key generation due to improper modular arithmetic.

B. Optimize exponentiation operations for large prime values.

C. Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

ANSWERS:

1. Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)

A Python program implementing **DES and 3DES encryption** produces inconsistent ciphertext for identical inputs across multiple runs.

A. Identify and fix logical and implementation errors in key scheduling and permutation steps.

DES (Data Encryption Standard) is a symmetric encryption algorithm that uses a 56-bit effective key and encrypts data in 64-bit blocks. It uses an initial permutation, sixteen rounds of encryption, and a final permutation.

Common errors in DES implementation include incorrect permutation tables, wrong bit positions, incorrect key scheduling, and improper left and right shifts.

Methods to fix the errors:

1. Check the initial and final permutation tables carefully.
2. Generate sixteen round keys using the correct key scheduling process.
3. Apply the correct circular shifts to the key halves.
4. Use the round keys in reverse order during decryption.
5. Test the program using known plaintext and ciphertext values.

In 3DES, encryption and decryption operations must be performed in the correct order using the selected keys.

B. Optimize the code to reduce execution time for large input datasets (≥10 MB).

The performance of DES and 3DES can be improved using the following methods:

1. Use optimized libraries: Use established cryptographic libraries instead of slow, bit-by-bit Python implementations.
2. Process data in chunks: Divide large files into smaller blocks to reduce memory usage.
3. Avoid repeated calculations: Calculate the key schedule once when the same key is used for multiple blocks.
4. Reduce memory copies: Avoid unnecessary conversions between strings, lists, and byte arrays.
5. Measure performance: Compare execution time before and after optimization using the same input data.

These methods reduce unnecessary computations and improve memory efficiency.

C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

Yes, replacing DES/3DES with AES improves security and can improve performance.

DES uses a 56-bit effective key, making it vulnerable to brute-force attacks. 3DES provides better protection than DES but is slower and is being phased out.

AES (Advanced Encryption Standard) supports 128-bit, 192-bit, and 256-bit keys. It provides stronger security and is efficiently implemented on modern processors. AES-GCM also provides authentication along with encryption.

Therefore, AES-GCM is a better choice for modern applications because it provides confidentiality, integrity, and good performance.

PYTHON CODE:

```
from Crypto.Cipher import DES3 from

Crypto.Random import get_random_bytes key

=

DES3.adjust_key_parity(get_random_bytes(24

)) data = b"Hello World" cipher =

DES3.new(key, DES3.MODE_EAX)

ciphertext, tag =

cipher.encrypt_and_digest(data)

print("Encrypted:", ciphertext.hex()) decipher =

DES3.new(     key, DES3.MODE_EAX,

nonce=cipher.nonce

)

plaintext = decipher.decrypt_and_verify(ciphertext, tag)

print("Decrypted:", plaintext.decode())
```

OUTPUT:

```python
from Crypto.Cipher import DES3
from Crypto.Random import get_random_bytes

key = DES3.adjust_key_parity(get_random_bytes(24))
data = b"Hello World"

cipher = DES3.new(key, DES3.MODE_EAX)
ciphertext, tag = cipher.encrypt_and_digest(data)

print("Encrypted:", ciphertext.hex())

decipher = DES3.new(
    key, DES3.MODE_EAX, nonce=cipher.nonce
)
plaintext = decipher.decrypt_and_verify(ciphertext, tag)

print("Decrypted:", plaintext.decode())
```

```
Encrypted: 84fe1550a92ad56af82f04
Decrypted: Hello World
```

4. **Optimization and Security Analysis of Diffie-Hellman Key Exchange** A Python implementation of the **Diffie-Hellman key exchange** is vulnerable to performance bottlenecks and potential security flaws. .

A. Debug incorrect shared-key generation due to improper modular arithmetic.

Diffie–Hellman is a key-exchange algorithm that allows two parties to establish a shared secret over an insecure communication channel.

Both parties agree on a prime number p and a generator g.

Step 1: Alice Generates Her Public Key

Alice selects a private key a and calculates her public key.

Formula:

$$A = (g^a) \bmod p$$

Step 2: Bob Generates His Public Key

Bob selects a private key b and calculates his public key.

Formula:

B = (g^b) mod p

Step 3: Alice Calculates the Shared Secret

Alice uses Bob's public key and her private key to calculate the shared secret.

Formula:

K_A = (B^a) mod p

Step 4: Bob Calculates the Shared Secret

Bob uses Alice's public key and his private key to calculate the shared secret.

Formula:

K_B = (A^b) mod p

Both parties obtain the same shared secret because:

K_A = (g^b)^a mod p

K_A = (g^(ab)) mod p

Similarly,

K_B = (g^a)^b mod p

K_B = (g^(ab)) mod p

Therefore, the final shared secret is:

K_A = K_B = (g^(ab)) mod p

Common Errors and Solutions

1. Incorrect modular arithmetic: Use modular exponentiation instead of calculating large powers directly.

2. Different parameters: Ensure both parties use the same valid prime number p generator g.

3. Weak private keys: Generate private keys using a cryptographically secure random number generator.

4. Invalid public values: Validate received public values according to the selected cryptographic group.

5. Incorrect shared-key calculation: Each party must use its own private key and the other party's public key.

B. Optimize exponentiation operations for large prime values.

Diffie–Hellman performance can be improved using fast modular exponentiation and trusted cryptographic libraries.

1. Fast Modular Exponentiation

Fast modular exponentiation calculates a power modulo p without calculating the complete large power first.

Formula for Alice's public key:

A = (g^a) mod p

Formula for Bob's public key:

B = (g^b) mod p

Formula for Alice's shared secret:

K_A = (B^a) mod p

Formula for Bob's shared secret:

K_B = (A^b) mod p

2. Other Optimization Methods

1. Use standardized cryptographic groups instead of designing custom parameters.

2. Avoid repeatedly generating expensive parameters when a standardized group can safely be reused.

C. Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

A basic Diffie–Hellman exchange does not verify the identities of the participants. An attacker may intercept and replace public values, creating separate shared secrets with Alice and Bob. This is known as a man-in-the-middle (MITM) attack.

Methods to Prevent MITM Attacks

1. Digital Certificates

Digital certificates help verify the identities of the participants before establishing secure communication.

2. Digital Signatures

Digitally sign the exchanged public values and verify the signatures before accepting them.

3. Authenticated Protocols

Use secure, authenticated protocols such as TLS to protect communication and authenticate participants.

4. Ephemeral Diffie–Hellman

Use ephemeral Diffie–Hellman methods, such as ECDHE, where supported. When properly authenticated, these methods can provide forward secrecy.

5. Key Derivation Function (KDF)

Use a secure key-derivation function to derive session keys from the shared secret.

Formula for the shared secret:

$K = (g^{(ab)}) \bmod p$

Formula for the session key:

K_session = KDF(K)

The raw shared secret should not be used directly as an encryption key. A secure KDF should incorporate the appropriate protocol context.

Performance Impact

Authentication adds computational overhead because digital signatures must be generated and verified, and certificates may need to be validated. This can increase processing time and communication overhead.

However, this additional cost is justified because authentication helps prevent unauthorized interception and public-key substitution.

PYTHON CODE:

p = 23

g = 5 a

= 6

b = 15

A = pow(g, a, p) B = pow(g, b, p)

key1 = pow(B, a, p) key2 = pow(A,

b, p) print("Alice public key:", A)

print("Bob public key:", B)

print("Alice shared key:", key1)

print("Bob shared key:", key2)

print("Keys match:", key1 == key2)

OUTPUT:

```
p = 23
g = 5

a = 6
b = 15

A = pow(g, a, p)
B = pow(g, b, p)

key1 = pow(B, a, p)
key2 = pow(A, b, p)

print("Alice public key:", A)
print("Bob public key:", B)
print("Alice shared key:", key1)
print("Bob shared key:", key2)
print("Keys match:", key1 == key2)
```

```
Alice public key: 8
Bob public key: 19
Alice shared key: 2
Bob shared key: 2
Keys match: True
```

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated

Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation